Programs as Data
A First-Order Functional Language

Peter Sestoft
Niels Hallenberg
2026-07-24

Plan for today

- A first-order functional language
  - Lexer and parser specifications
  - Interpretation: function closures
- Explicit types
  - Static versus dynamic type
  - Type checking function
  - Type rules and derivations

The overall course plan

- Abstract syntax
- Interpreting an expression language
- Lexing and parsing tools
- Interpreting a functional language, micro-ML
  - Higher-order functions
- Type checking and type inference
- Interpreting an imperative language, micro-C
- Compiling micro-C to stack machine code
- Real-world abstract machines: JVM and .NET
- Garbage collection techniques
- Continuations, exceptions and backtracking
- A locally optimizing compiler
- Compiling micro-ML to stack machine code
- Typing micro-Java
- Compiling micro-Java
- Compiling to assembly

Micro-ML: A small functional language

- First-order: A value cannot be a function
- Dynamically typed, so this is OK:

```
if true then 1+2 else 1+false
```

- Eager, or call-by-value: In a call `f(e)` the argument `e` is evaluated before `f` is called
- Example Micro-ML programs (an F# subset):

```
5+7

let f x = x + 7 in f 2 end

let fac x = if x=0 then 1 else x * fac(x - 1)
in fac 10 end
```

Abstract syntax of Micro-ML

```
type expr =
  | CstI of int
  | CstB of bool
  | Var of string
  | Let of string * expr * expr
  | Prim of string * expr * expr
  | If of expr * expr * expr
  | Letfun of string * string * expr * expr
  | Call of expr * expr
```

```
let f x = x + 7 in f 2 end
```

(f, x, fBody, letBody)

```
Letfun ("f", "x", Prim ("+",Var "x",CstI 7),
        Call (Var "f",CstI 2))
```

Absyn.fs

Runtime values, function closures

- Run-time values: integers and functions

```
type value =
  | Int of int
  | Closure of string * string * expr * value env
```

- *Closure*: a package of a function's body and its declaration environment
- A name should refer to a *statically* enclosing binding:

```
let y = 11
in let f x = x + y
   in let y = 22 in f 3 end
   end
end
```

Should always have value 11

Evaluate as 3 + y

(f, x, x+y, [(y,11)])

Fun.fs

Interpretation of Micro-ML

- Constants, variables, primitives, let, if: as for expressions
- Letfun: Create function closure and bind f to it
- Function call f(e):
  - Look up f, it must be a closure
  - Evaluate e
  - Create environment and evaluate the function's body

```
let rec eval (e : expr) (env : value env) : int =
    match e with
    | ...
    | Letfun(f, x, fBody, letBody) ->
      let bodyEnv = (f, Closure(f, x, fBody, env)) :: env
      eval letBody bodyEnv
    | Call(Var f, eArg) ->
      let fClosure = lookup env f
      match fClosure with
          | Closure (f, x, fBody, fDeclEnv) ->
            let xVal = Int(eval eArg env)
            let fBodyEnv = (x, xVal) :: (f, fClosure) :: fDeclEnv
            eval fBody fBodyEnv
          | _ -> failwith "eval Call: not a function"
```

Closure

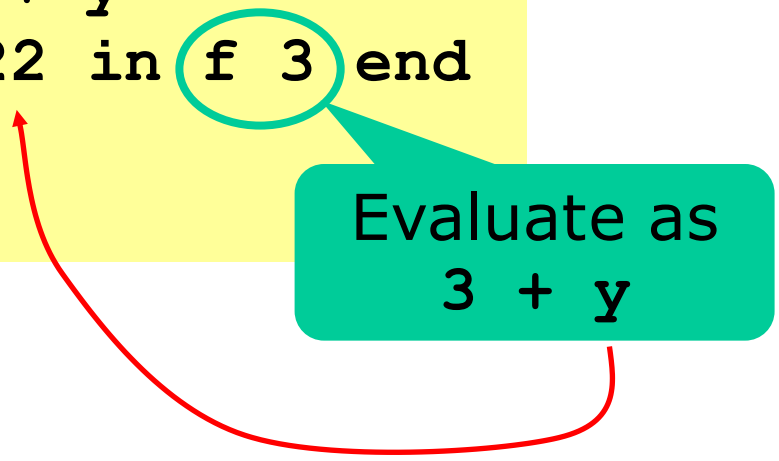
Evaluate fBody in declaration environment

Fun.fs

Dynamic scope (instead of static)

- With static scope, a variable refers to the lexically, or statically, most recent binding
- With **dynamic scope**, a variable refers to the dynamically most recent binding:

```
let y = 11
in let f x = x + y
   in let y = 22 in f 3 end
   end
end
```



A dynamic scope variant of Micro-ML

- Very minimal change in interpreter:

```
let rec eval (e : expr) (env : value env) : int =
    ...
    | Call(Var f, eArg) ->
      let fClosure = lookup env f
      in match fClosure with
         | Closure (f, x, fBody, fDeclEnv) ->
           let xVal = Int(eval eArg env)
           let fBodyEnv = (x, xVal) :: (f, fClosure) :: env
           in eval fBody fBodyEnv
```

Evaluate fBody in call environment

- fDeclEnv is ignored; function is just (f, x, fBody)
- Good and bad:
  - simple to implement (no closures needed)
  - makes type checking difficult
  - makes efficient implementation difficult
- Used in macro languages, and Lisp, Perl, Clojure

Lexer and parser for Micro-ML

- Lexer:
  - Nested comments, as in F#, Standard ML

```
1 + (* 33 (* was 44 *) *) 22
```

- Parser:
  - To parse applications $e_1\ e_2\ e_3$ correctly, distinguish atomic expressions from others

```
if 2 < 3 then 4 else f 2 3
```

non-atomic!

- Problem: f(x-1) parses as f(x(-1))
- Solution:
  - FunLex.fsl: make **`CSTINT`** just **`[0-9]+`** without sign
  - FunPar.fsy: add rule **`Expr := MINUS Expr`**

Plan for today

- A first-order functional language
  - Lexer and parser specifications
  - Interpretation: function closures
- **Explicit types**
  - **Static versus dynamic type**
  - **Type checking function**
  - **Type rules and derivations**

An explicitly typed fun. language

```
let f (x : int) : int = x+1
in f 12 end
```

```
type typ =
   | TypI
   | TypB
   | TypF of typ * typ
```

```
Letfun("f", "x", TypI,
       Prim("+", Var "x", CstI 1), TypI,
       Call(Var "f", CstI 12));;
```

(TypF(TypI, TypI)

```
type tyexpr =
   | CstI of int
   | CstB of bool
   | Var of string
   | Let of string * tyexpr * tyexpr
   | Prim of string * tyexpr * tyexpr
   | If of tyexpr * tyexpr * tyexpr
   | Letfun of string * string * typ * tyexpr * typ * tyexpr
   | Call of tyexpr * tyexpr
```

(f, x, xTyp, fBody, rTyp, letBody

Type systems

- 'Well-typed programs can't go wrong' – Robin Milner
  - Ensure that our programs can not perform invalid operations such as adding a boolean to an integer
- The power of the dot
  - Smart code completion, code generation and semantic highlighting – better IDE support
- Lightweight contract mechanism
  - States what types of input is expected explicitly rather than requiring the programmer to guess
- Optimizations
  - Statically typed languages usually can apply more types of optimizations

Type checking by recursive function

- Using a type environment [("x", TypI)]:

```
let rec typ (e : tyexpr) (env : typ env) : typ =
    match e with
    | CstI i -> TypI
    | CstB b -> TypB
    | Var x  -> lookup env x
    | Prim(ope, e1, e2) ->
      let t1 = typ e1 env
      let t2 = typ e2 env
      in match (ope, t1, t2) with
          | ("*",  TypI, TypI) -> TypI
          | ("+",  TypI, TypI) -> TypI
          | ("-",  TypI, TypI) -> TypI
          | ("=",  TypI, TypI) -> TypB
          | ("<",  TypI, TypI) -> TypB
          | ("&&", TypB, TypB) -> TypB
          | _    -> failwith "unknown primitive, or type error"
    | ...
```

Type checking, part 2

- Checking `let x=eRhs in letBody end`
- Checking `if e1 then e2 else e3`

```
let rec typ (e : tyexpr) (env : typ env) : typ =
    match e with
    | Let(x, eRhs, letBody) ->
      let xTyp = typ eRhs env
      let letBodyEnv = (x, xTyp) :: env
      in typ letBody letBodyEnv
    | If(e1, e2, e3) ->
      match typ e1 env with
       | TypB -> let t2 = typ e2 env
                 let t3 = typ e3 env
                 in if t2 = t3 then t2
                    else failwith "If: branch types differ"
       | _    -> failwith "If: condition not boolean"
    | ...
```

Type checking, part 3

- Checking `let f x=eBody in letBody end`
- Checking `f eArg`

```
let rec typ (e : tyexpr) (env : typ env) : typ =
    match e with
    | ...
    | Letfun(f, x, xTyp, fBody, rTyp, letBody) ->
      let fTyp = TypF(xTyp, rTyp)
      let fBodyEnv = (x, xTyp) :: (f, fTyp) :: env
      let letBodyEnv = (f, fTyp) :: env
      if typ fBody fBodyEnv = rTyp then typ letBody letBodyEnv
      else failwith "Letfun: wrong return type in function"
    | Call(Var f, eArg) ->
      match lookup env f with
       | TypF(xTyp, rTyp) ->
         if typ eArg env = xTyp then rTyp
         else failwith "Call: wrong argument type"
       | _ -> failwith "Call: unknown function"
    | Call(_, eArg) -> failwith "Call: illegal function in call"
```

Type checking versus evaluation

- The type checker **`typ`** and the interpreter **`eval`** have similar structure
- Type checking can be thought of as *abstract interpretation* of the program
- We calculate "TypI + TypI gives TypI" instead of "Int 3 + Int 5 gives Int 8"
- One major difference:
  - Type checking a function call f(e) does not require type checking the function's body again
  - Interpreting a function call f(e) does require interpreting the function's body
- Type checking always terminates

Type checking by logical rules

$$\frac{}{\rho \vdash i : \text{int}}\,(\text{t1}) \qquad \frac{}{\rho \vdash b : \texttt{bool}}\,(\text{t2})$$

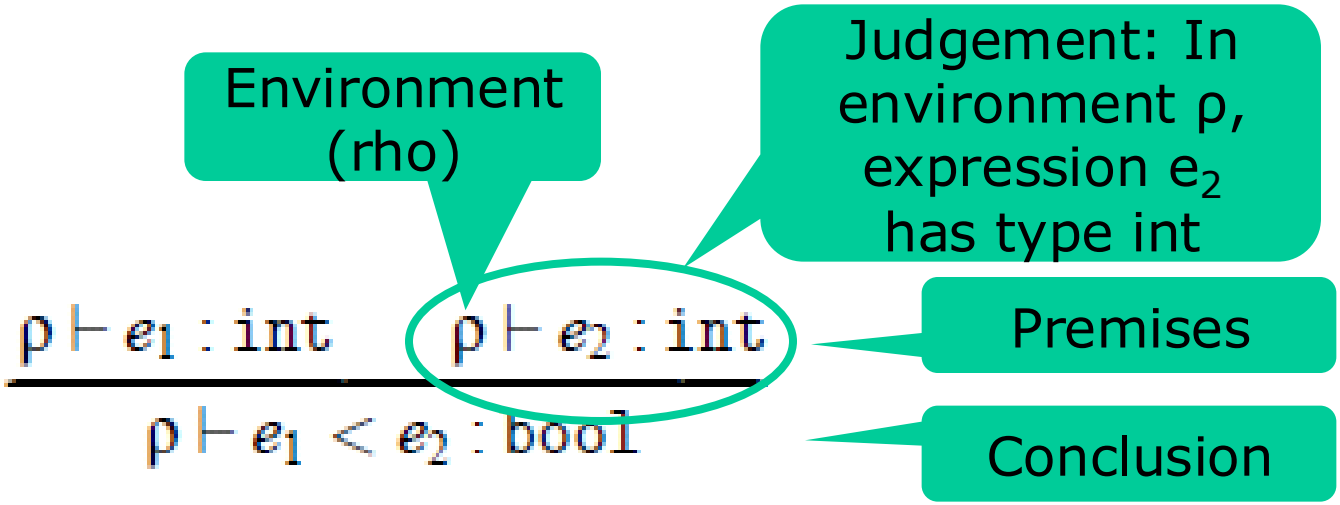
$$\frac{\rho(x) = t}{\rho \vdash x : t}\,(\text{t3}) \qquad \frac{\rho \vdash e_1 : \texttt{int} \quad \rho \vdash e_2 : \texttt{int}}{\rho \vdash e_1 + e_2 : \texttt{int}}\,(\text{t4})$$

$$\frac{\rho \vdash e_1 : \texttt{int} \quad \rho \vdash e_2 : \texttt{int}}{\rho \vdash e_1 < e_2 : \texttt{bool}}\,(\text{t5}) \qquad \frac{\rho \vdash e_r : t_r \quad \rho[x \mapsto t_r] \vdash e_b : t}{\rho \vdash \texttt{let}\ x = e_r\ \texttt{in}\ e_b\ \texttt{end} : t}\,(\text{t6})$$

$$\frac{\rho \vdash e_1 : \texttt{bool} \quad \rho \vdash e_2 : t \quad \rho \vdash e_3 : t}{\rho \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : t}\,(\text{t7}) \qquad \frac{\rho(f) = t_x \to t_r \quad \rho \vdash e : t_x}{\rho \vdash f\ e : t_r}\,(\text{t9})$$

$$\frac{\rho[x \mapsto t_x, f \mapsto t_x \to t_r] \vdash e_r : t_r \quad \rho[f \mapsto t_x \to t_r] \vdash e_b : t}{\rho \vdash \texttt{let}\ f\ (x : t_x) = e_r\ \texttt{in}\ e_b\ \texttt{end} : t}\,(\text{t8})$$

How to read a type rule



- IF
  - in environment ρ, expression e_1 has type int, and
  - in environment ρ, expression e_2 has type int
- THEN
  - in environment ρ, expression $e_1 < e_2$ has type bool

Environments

- Type environments map variables to their types
- Operations:

$\rho(x)$	Look up x in ρ
$\rho[x \mapsto t]$	Extend ρ with x mapping to type t
$\rho[x_0 \mapsto t_0, \ldots, x_n \mapsto t_n]$	Extend with multiple variables/types

- Examples:

$[]$	The empty environment
$[x \mapsto \texttt{int}, y \mapsto \texttt{bool} \to \texttt{int}]$	A concrete environment

$\rho[x \mapsto t](x) = t$	Looking up in an extended environment
$\rho[x \mapsto t](y) = \rho(y)$	Looking up in an extended environment (assuming $x \neq y$)

Joint exercise: How read these?

$$\frac{}{\rho \vdash i : \mathrm{int}}$$

An integer constant has type int

$$\frac{\rho(x) = t}{\rho \vdash x : t}$$

$$\frac{\rho \vdash e_1 : \mathtt{bool} \qquad \rho \vdash e_2 : t \qquad \rho \vdash e_3 : t}{\rho \vdash \mathtt{if}\ e_1\ \mathtt{then}\ e_2\ \mathtt{else}\ e_3 : t}$$

$$\frac{\rho \vdash e_r : t_r \qquad \rho[x \mapsto t_r] \vdash e_b : t}{\rho \vdash \mathtt{let}\ x = e_r\ \mathtt{in}\ e_b\ \mathtt{end} : t}$$

Combining type rules to trees

- Stacking type rules on top of each other
- One rule's conclusion is another's premise
- Checking **`let x=1 in x<2 end : bool`** in some environment ρ:

$$\frac{\dfrac{}{\rho \vdash 1 : \mathtt{int}} \qquad \dfrac{\rho[x \mapsto \mathtt{int}] \vdash x : \mathtt{int} \qquad \rho[x \mapsto \mathtt{int}] \vdash 2 : \mathtt{int}}{\rho[x \mapsto \mathtt{int}] \vdash x < 2 : \mathtt{bool}}}{\rho \vdash \mathtt{let}\ x\ =\ 1\ \mathtt{in}\ x < 2\ \mathtt{end} : \mathtt{bool}}$$

- The **`typ`** function implements the rules, from conclusion to premise!

Type checking example expression

Open your book on page 71

$$\frac{\vdots}{\rho \vdash \texttt{let}\ x = 1\ \texttt{in}\ x < 2\ \texttt{end} : \texttt{bool}}$$

Type checking example expression

$$\frac{\rho \vdash e_r : t_r \qquad \rho[x \mapsto t_r] \vdash e_b : t}{\rho \vdash \texttt{let}\ x = e_r\ \texttt{in}\ e_b\ \texttt{end} : t}$$

$$\frac{\dfrac{\vdots}{\rho \vdash 1 : \texttt{int}} \qquad \dfrac{\vdots}{\rho[x \mapsto \texttt{int}] \vdash x < 2 : \texttt{bool}}}{\rho \vdash \texttt{let}\ x = 1\ \texttt{in}\ x < 2\ \texttt{end} : \texttt{bool}}\ (\text{t6})$$

Type checking example expression

$$\frac{}{\rho \vdash i : \mathtt{int}}$$

$$\frac{\dfrac{}{\rho \vdash 1 : \mathtt{int}}(\mathrm{t1}) \qquad \dfrac{\vdots}{\rho[x \mapsto \mathtt{int}] \vdash x < 2 : \mathtt{bool}}}{\rho \vdash \mathtt{let}\ x = 1\ \mathtt{in}\ x < 2\ \mathtt{end} : \mathtt{bool}}(\mathrm{t6})$$

Type checking example expression

$$\frac{\rho \vdash e_1 : \texttt{int} \qquad \rho \vdash e_2 : \texttt{int}}{\rho \vdash e_1 < e_2 : \texttt{bool}}$$

$$\dfrac{\dfrac{}{\rho \vdash 1 : \texttt{int}}(\text{t1}) \qquad \dfrac{\dfrac{\vdots}{\rho[x \mapsto \texttt{int}] \vdash x : \texttt{int}} \qquad \dfrac{\vdots}{\rho[x \mapsto \texttt{int}] \vdash 2 : \texttt{int}}}{\rho[x \mapsto \texttt{int}] \vdash x < 2 : \texttt{bool}}(\text{t5})}{\rho \vdash \texttt{let } x = 1 \texttt{ in } x < 2 \texttt{ end} : \texttt{bool}}(\text{t6})$$

Type checking example expression

$$\frac{\rho(x) = t}{\rho \vdash x : t}$$

$$\dfrac{\dfrac{}{\rho \vdash 1 : \texttt{int}}(\text{t1}) \quad \dfrac{\dfrac{\rho[x \mapsto \texttt{int}](x) = \texttt{int}}{\rho[x \mapsto \texttt{int}] \vdash x : \texttt{int}}(\text{t3}) \quad \dfrac{\vdots}{\rho[x \mapsto \texttt{int}] \vdash 2 : \texttt{int}}}{\rho[x \mapsto \texttt{int}] \vdash x < 2 : \texttt{bool}}(\text{t5})}{\rho \vdash \texttt{let } x = 1 \texttt{ in } x < 2 \texttt{ end} : \texttt{bool}}(\text{t6})$$

Type checking example expression

$$\frac{}{\rho \vdash i : \mathtt{int}}$$

$$\dfrac{\dfrac{}{\rho \vdash 1 : \mathtt{int}}(\mathrm{t1}) \quad \dfrac{\dfrac{\rho[x \mapsto \mathtt{int}](x) = \mathtt{int}}{\rho[x \mapsto \mathtt{int}] \vdash x : \mathtt{int}}(\mathrm{t3}) \quad \dfrac{}{\rho[x \mapsto \mathtt{int}] \vdash 2 : \mathtt{int}}(\mathrm{t1})}{\rho[x \mapsto \mathtt{int}] \vdash x < 2 : \mathtt{bool}}(\mathrm{t5})}{\rho \vdash \mathtt{let}\ x = 1\ \mathtt{in}\ x < 2\ \mathtt{end} : \mathtt{bool}}(\mathrm{t6})$$

Notice

The `typ` function implements the rules, from conclusion to premise!

Common Pitfalls

- Things that are written the same in the rule must have the same value in a derivation
  - e.g., ρ must be the same for the whole rule, even though we have extended it before
  - e.g., each place where it has t_x then the same value must be written
- t is not a concrete type and should often not be in the final derivation. int, bool, int -> bool, ... are all concrete types
- Remember to do a variable lookup in the environment when using rule t3 when checking that a variable has a specific type.

Joint exercises: Invent type rules

- For e_1 `&&` e_2 (logical and)
- For e_1 `::` e_2 (list cons operator)
- For `match e with []` `->` e_1 `|` `x::xr` `->` e_2

Evaluation Rules

$$\frac{}{\rho \vdash i \Rightarrow i}\ (e1)$$

$$\frac{}{\rho \vdash b \Rightarrow b}\ (e2)$$

$$\frac{\rho(x) = v}{\rho \vdash x \Rightarrow v}\ (e3)$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \qquad \rho \vdash e_2 \Rightarrow v_2 \qquad v = v_1 + v_2}{\rho \vdash e_1 + e_2 \Rightarrow v}\ (e4)$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \qquad \rho \vdash e_2 \Rightarrow v_2 \qquad b = (v_1 < v_2)}{\rho \vdash e_1 < e_2 \Rightarrow b}\ (e5)$$

$$\frac{\rho \vdash e_r \Rightarrow v_r \qquad \rho[x \mapsto v_r] \vdash e_b \Rightarrow v}{\rho \vdash \texttt{let}\ x = e_r\ \texttt{in}\ e_b\ \texttt{end} \Rightarrow v}\ (e6)$$

$$\frac{\rho \vdash e_1 \Rightarrow \texttt{true} \qquad \rho \vdash e_2 \Rightarrow v}{\rho \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \Rightarrow v}\ (e7t)$$

$$\frac{\rho \vdash e_1 \Rightarrow \texttt{false} \qquad \rho \vdash e_3 \Rightarrow v}{\rho \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \Rightarrow v}\ (e7f)$$

$$\frac{\rho[f \mapsto (f, x, e_r, \rho)] \vdash e_b \Rightarrow v}{\rho \vdash \texttt{let}\ f\ (x) = e_r\ \texttt{in}\ e_b\ \texttt{end} \Rightarrow v}\ (e8)$$

$$\frac{\rho(f) = (f, x, e_r, \rho_{fdecl}) \qquad \rho \vdash e \Rightarrow v_x \qquad \rho_{fdecl}[x \mapsto v_x, f \mapsto (f, x, e_r, \rho_{fdecl})] \vdash e_r \Rightarrow v}{\rho \vdash f\ e \Rightarrow v}\ (e9)$$

Fig. 4.3 Evaluation rules for a first-order functional language.

Constants

Look up variable in environment

Rules for simple operators

Bind x to result of evaluating e_r in e_b where x is in scope

Evaluation of e_1 decides whether rule *e7t* or *e7f* is used.

Evaluate e_b with env ρ extended with f bound to closure for f.

f must be a closure. Evaluate body of f, e_r, with env ρ extended with f bound to own closure. We allow f to call itself.

Evaluation by logical rules: Function declaration and call

- Compare these with the **`eval`** interpreter:

$$\frac{\rho[f \mapsto (f, x, e_r, \rho)] \vdash e_b \Rightarrow v}{\rho \vdash \texttt{let } f\ (x) = e_r \texttt{ in } e_b \texttt{ end} \Rightarrow v}\ (e8)$$

$$\frac{\rho(f) = (f, x, e_r, \rho_{fdecl}) \qquad \rho \vdash e \Rightarrow v_x \qquad \rho_{fdecl}[x \mapsto v_x, f \mapsto (f, x, e_r, \rho_{fdecl})] \vdash e_r \Rightarrow v}{\rho \vdash f\ e \Rightarrow v}\ (e9)$$



- Also, note recursive evaluation of f's body; no such thing in the type rules

Joint exercises

- Draw the evaluation trees for some of these

```
let x = 1
in x < 2
```

```
let f x = 1
in f 2 + f false
```

$$\frac{}{\rho \vdash i \Rightarrow i}\ (e1)$$

$$\frac{}{\rho \vdash b \Rightarrow b}\ (e2)$$

$$\frac{\rho(x) = v}{\rho \vdash x \Rightarrow v}\ (e3)$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \qquad \rho \vdash e_2 \Rightarrow v_2 \qquad v = v_1 + v_2}{\rho \vdash e_1 + e_2 \Rightarrow v}\ (e4)$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \qquad \rho \vdash e_2 \Rightarrow v_2 \qquad b = (v_1 < v_2)}{\rho \vdash e_1 < e_2 \Rightarrow b}\ (e5)$$

$$\frac{\rho \vdash e_r \Rightarrow v_r \qquad \rho[x \mapsto v_r] \vdash e_b \Rightarrow v}{\rho \vdash \texttt{let}\ x = e_r\ \texttt{in}\ e_b\ \texttt{end} \Rightarrow v}\ (e6)$$

$$\frac{\rho \vdash e_1 \Rightarrow \texttt{true} \qquad \rho \vdash e_2 \Rightarrow v}{\rho \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \Rightarrow v}\ (e7t)$$

$$\frac{\rho \vdash e_1 \Rightarrow \texttt{false} \qquad \rho \vdash e_3 \Rightarrow v}{\rho \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \Rightarrow v}\ (e7f)$$

$$\frac{\rho[f \mapsto (f, x, e_r, \rho)] \vdash e_b \Rightarrow v}{\rho \vdash \texttt{let}\ f\ (x) = e_r\ \texttt{in}\ e_b\ \texttt{end} \Rightarrow v}\ (e8)$$

$$\frac{\rho(f) = (f, x, e_r, \rho_{fdecl}) \qquad \rho \vdash e \Rightarrow v_x \qquad \rho_{fdecl}[x \mapsto v_x, f \mapsto (f, x, e_r, \rho_{fdecl})] \vdash e_r \Rightarrow v}{\rho \vdash f\ e \Rightarrow v}\ (e9)$$

Fig. 4.3 Evaluation rules for a first-order functional language.

Dynamically or statically typed

- Dynamically typed:
  - Types are checked during evaluation (micro-ML, Postscript, JavaScript, Python, Ruby, Scheme, …)

```
true { 11 } { 22 false add } ifelse =
```

OK, gives 11

- Statically typed:
  - Types are checked before evaluation (our typed fun. language, F#, most of Java and C#)

```
if true then 11 else 22+false
```

Compile-time type error

```
true ? 11 : (22 + false)
```

Compile-time type error

Dynamic typing in Java/C# arrays

- For a Java/C# array whose element type is a reference type, all assignments are type-checked at runtime

```
void M(Object[] arr, Object x) {
  arr[0] = x;
}
```

Type check needed at run-time

- Why is that necessary?

```
String[] ss = new String[1];
M(ss, new Object());
String s0 = ss[0];
```